

Overview

`detectCommunities` in `packages/graph-view/src/utils/louvain.ts` was updated to handle duplicate node IDs more explicitly and to guard against accidental map overwrites. The function signature and return type remain unchanged.

Key changes

- **Explicit deduplication** – `const uniqueIds = [...new Set(nodeIds)]` (R21–R22) replaces the previous `new Set(nodeIds)` (L21). Nodes are added with `for (const id of uniqueIds) g.addNode(id)` (R24, replacing L23).
- **Map population guard** – The loop that populates the result map now checks `if (!map.has(id))` before inserting (R35–R37, replacing L34). This prevents overwriting entries when `nodeIds` contains duplicates.
- **Comment clarification** – The JSDoc now notes that `graphology-communities-louvain` already assigns unique IDs to disconnected nodes, but the defensive reassignment of `-1` sentinels is retained (lines 10–15).
- **Variable naming** – `ids` is now built from `uniqueIds` (R22), improving readability.

Impact

- **Correctness** – Duplicate `nodeIds` are still handled correctly; the guard ensures each unique ID appears once in the returned map.
- **Performance** – The added array and Set construction is negligible for typical graph sizes.
- **Maintainability** – The intent of deduplication and defensive mapping is clearer, reducing the risk of accidental duplicate handling.

Risks & follow-ups

- **Duplicate node handling** – Verify that tests with duplicate `nodeIds` still pass and that the map contains a single entry per unique ID.
- **Empty `nodeIds`** – Ensure the function returns an empty map without errors.
- **Linting** – Run `npm run lint` to confirm no new style violations.
- **Performance regression** – Benchmark on large graphs to confirm the added overhead is insignificant.